Indian Institute of Technology Madras

CS6886: Systems Engineering for Deep Learning

Project Report

Exploring LLS: Granularity, Early Exit and Scaling LLS

Yukash Ram
CS23B069

Hayagriv Venkatesh
ED23B024

October 2026

Code: `https://github.com/Yukash09/CS6886_Project`

Abstract

LLS [1], a local learning rule inspired by neural activity synchronization, trains each layer with its own local loss against a fixed periodic basis, removing end-to-end backpropagation (BP) and most of its training memory at a cost in accuracy. We study four questions. For *granularity*, we group layers into blocks that share one LLS head and find that where the cuts go matters more than how many heads remain: merging the early layers closes 89% of the LLS–BP accuracy gap on VGG8 at 1.29× the per-layer LLS memory, and 94% on MobileNetV1 at 2.56× (BP needs 8.21×), while simple uniform or pooling-aligned rules do not transfer between models. For *early exit*, we use the LLS heads as free exits: within one point of the full network, MobileNetV1 runs at 10–15% of its compute and VGG8 at 57–68%, a gap set by the architecture rather than the stopping rule. Coarser partitions give better exits, fixed heads match learned linear exits except where the head input is very wide, and BP with jointly trained exits remains better at every cost. The per-layer heads are also interpretable: they are calibrated, they show where a network stops improving, and their agreement tracks how hard an input is. For *scalability*, grouping ResNet-20 into three resolution-aligned blocks improves on per-block LLS-MxM by 15.8 points, and a two-block Wide-ResNet-20 is 2.5 points below BP with 12.7% less memory. For *attention*, a six-block ViT-Tiny is 3.1 points below BP with 74% less memory.

1 Introduction

Training a network with backpropagation (BP) requires storing the activations of every layer until the backward pass, so training memory grows with depth and limits on-device learning. Local learning rules avoid this by training each layer with its own loss, and LLS [1] is a recent rule that does so with a fixed, parameter-free target per layer. The price is accuracy: per-layer LLS trails BP by several points. In this project we study how far that gap can be closed and what else the local heads can be used for. Specifically, we ask four questions:

1. **Granularity:** instead of one LLS head per layer, can a block of layers share one head, and which grouping gives the best accuracy for a given training memory?
2. **Early exit:** since every LLS head already produces a class prediction, can these predictions be used to stop inference early, and do they tell us anything interpretable about the network?
3. **Scalability:** does LLS scale beyond VGG8 and MobileNetV1 to deeper residual networks?
4. **Attention:** does LLS work on self-attention, i.e. on a Vision Transformer?

2 Prior Work

Local learning rules replace the back-propagated error with a signal that each layer can compute on its own. Feedback alignment and its variants send the error or the label through fixed random matrices [3, 4, 5], Forward-Forward replaces the backward pass with a second forward pass [6], and local-classifier methods attach a small trainable classifier to each block [7, 8]. The last family scales best to deep networks, but each classifier adds parameters and compute.

LLS [1] is a local learning rule inspired by *neural activity synchronization*: in the brain, neurons that respond to the same stimulus tend to fire together in a common rhythm. LLS imitates this by asking the activity of each layer to follow a periodic pattern whose frequency identifies the class, so the class is encoded in when neurons fire rather than in a learned output layer. In effect it keeps the idea of a local classifier but makes it fixed. Each layer l, with (pooled) output $h^{(l)} \in \mathbb{R}^{T_l}$, is assigned a basis $B^{(l)} \in \mathbb{R}^{C \times T_l}$ whose row c is a square wave with a class-specific frequency f_c. The layer is trained to make its output resemble the wave of the correct class:

$$\mathcal{L}^{(l)} = \mathrm{CE}\left(h^{(l)} B^{(l)\top},\, y\right), \qquad \delta^{(l)} = \left(p^{(l)} - y\right) B^{(l)}, \tag{1}$$

where $p^{(l)}$ is the softmax of the projection. Since waves of different frequencies are nearly orthogonal, the basis acts as a fixed classifier that separates the classes without being trained. Because the learning signal $\delta^{(l)}$ depends only on the layer's own output, no error crosses layers and only one layer's activations need to be kept at a time. We refer to the projection $h^{(l)} B^{(l)\top}$ as the *LLS head* of layer l. The paper also proposes two variants with a few learned parameters: LLS-M learns one scale per class, and LLS-MxM learns a $C \times C$ matrix that mixes the fixed class scores. The original work evaluates only VGG and MobileNet models; we use ResNets [13] for the scalability question and Vision Transformers [14] for the attention question. Early-exit networks [10, 11], which attach trained classifiers to intermediate layers to save inference compute, motivate our second question: LLS already provides such classifiers at no extra cost.

3 Implementation Details

3.1 Granularity

The reference implementation [2] fixes one head per layer, so we reimplemented it to make the head positions a parameter. A network is described as a list of *units*, each a trainable layer together with its pooling, and a *partition* such as $(2, 5)$ groups consecutive units into blocks. Every block gets one LLS head on its output and its own optimizer, which makes per-layer LLS $(1, \ldots, 1)$ and BP (L) two ends of the same model. During a training step, each block takes the detached output of the previous block, backpropagates its head loss only within itself, and updates; no gradient crosses a block boundary. The intuition is that a per-layer head forces every layer to be a good classifier on its own, while inside a block

BP lets several layers cooperate on building features, and the memory grows only to that of the largest block. Peak training memory is therefore set by the largest block, which we measure exactly by counting, with PyTorch's saved-tensor hooks, the bytes that autograd keeps for each block's backward pass.

With this setup we trained all $2^6 = 64$ partitions of VGG8. MobileNetV1 has 2^{13} partitions, too many to sweep, so we chose 14 that test the patterns seen on VGG8: uniform blocks, cuts at pooling layers, and merges at the start or end of the network. We also checked seed variance on VGG8, repeated the key partitions on CIFAR100, and tested whether short 10-epoch runs are enough to choose a partition. All runs use schedule-free AdamW [15].

3.2 Early Exit with LLS Heads

Every LLS head is trained to classify, so a granular LLS network is already a multi-exit network: an exit costs only one projection onto the fixed basis and needs no extra training. For each trained network we record the logits of every head on the validation and test sets once, so that stopping rules can be compared offline without rerunning the model. A sample exits at the first head where the rule fires; its cost is the backbone MACs up to that head plus the cost of every head evaluated. We implemented the standard rules (confidence, entropy, margin and patience) and a *sync* rule of our own, which exits only when the last w heads agree on the class distribution:

$$\frac{2}{w(w-1)} \sum_{k-w<i<j\leq k} \cos(p_i, p_j) \;\geq\; \tau. \tag{2}$$

The intuition follows from how LLS trains: every layer is pushed towards the same class-specific pattern, so once consecutive heads give the same distribution, the representation has settled and deeper layers are unlikely to change the decision. Unlike confidence, which trusts a single head, sync requires agreement across depth, so one overconfident head cannot trigger an exit; and unlike patience, which only compares the top class, it compares full distributions, so the heads must also agree on how uncertain they are. For each rule we select, on validation, the cheapest threshold whose accuracy stays within $\delta = 1\%$ of the full network, and report that setting on test. To judge whether fixed heads are good exits, we compare them with linear exits trained on the same frozen features, and with BP networks whose exits are trained either after the backbone (frozen) or together with it (joint).

3.3 Scalability: ResNet

We apply the same partition code to residual networks, with one change in what counts as a unit: a unit is a complete basic block, including both convolutions, the shortcut projection and the residual addition, so that a block boundary never cuts through a skip connection; cutting one would detach the identity path that makes deep ResNets trainable. We support ResNet-20/32/56 and a Wide-ResNet-20 with doubled channels (32/64/128), with fixed LLS, LLS-M or LLS-MxM heads, and an option to predict with the last LLS head instead of a detached linear classifier. Resolution-aligned partitions follow the stages of the network, where the feature hierarchy changes naturally, and the wider network gives each local block more capacity to fit its own loss. On ResNet-20 we compare BP, per-block LLS and LLS-MxM, and a resolution-aligned $(4, 3, 3)$ partition with LLS-MxM. On Wide-ResNet-20 we compare BP with a two-block $(7, 3)$ LLS-MxM model that predicts with its last LLS head.

3.4 Attention: ViT

We implemented a CIFAR ViT-Tiny with 12 pre-norm encoder blocks, 192-dimensional tokens, three heads and 4×4 patches. A unit is one encoder block with both of its residual branches; the first unit also holds the patch embedding, CLS token and positional embeddings. Since an encoder block outputs tokens rather than a feature map, the LLS head projects the concatenation of the CLS token and the mean patch token: the CLS token only gathers class information in later blocks, while the mean patch token gives a stable summary from the first block on. We compare BP with a six-block $(2, 2, 2, 2, 2, 2)$ LLS-MxM model, both trained for 100 epochs on the same GPU. All ResNet and ViT runs use CIFAR10 with the same data split, batch size and schedule-free AdamW as the rest of the project, with model selection on validation accuracy.

4 Evaluation

4.1 Granularity

We compare each partition with two baselines: per-layer LLS, which has the lowest memory, and BP, which has the highest accuracy (Table 1). On VGG8, $(2, 5)$ improves on per-layer LLS by 5.4 points (91.6% vs 86.2%) at 1.29× its memory, and is 0.7 points below BP with 40% less memory than BP. On MobileNetV1, $(4, 2, 1 \times 8)$ improves on per-layer LLS by 15.2 points (83.0% vs 67.8%) and is 1.0 point below BP with 69% less memory than BP. Both winners merge the first layers; merging the last four layers of MobileNetV1 is 1.1 points worse than per-layer LLS. We attribute this to the per-layer heads forcing early layers to separate classes before they have built useful features. Partitions with equal memory differ by 2–3 points on VGG8 (seed std 0.1–0.3), and uniform or pooling-aligned blocks are 2–2.5 points below the best, so the partition has to be searched per model; ranking partitions after 10 epochs finds the best one in 4 of 5 memory budgets.

Table 1: Granularity on CIFAR10. G: share of the LLS–BP accuracy gap closed; Mem.: peak training memory relative to per-layer LLS.

VGG8 (60 epochs)				MobileNetV1 (300 epochs)			
Partition	Acc.	G	Mem.	Partition	Acc.	G	Mem.
LLS 1×7	86.2	0%	1.00	LLS 1×14	67.8	0%	1.00
Uniform $(2, 2, 2, 1)$	89.4	53%	1.29	Merge last 4	66.7	−7%	2.37
$(1, 1, 5)$	89.9	60%	1.00	Uniform 2×7	77.3	58%	1.54
$(2, 5)$	91.6	89%	1.29	$(4, 2, 1 \times 8)$	83.0	94%	2.56
BP	92.3	100%	2.15	BP	84.0	100%	8.21

4.2 Early Exit

We compare the LLS exits with running the full network, with learned exits, and with BP (Table 2). Against the full network, the LLS exits cut inference MACs by 85–90% on MobileNetV1 and by 32–43% on VGG8 at a cost of at most one point of accuracy. VGG8 saves less because 42% of its MACs are in full-resolution early convolutions that come before its first useful exit. Against per-layer LLS, the coarser partition $(4, 2, 6, 2)$ gives much better exits on MobileNetV1: 82.3% at 10% of the MACs, versus 67.2% at 12%. Against linear exits trained on the same features, the fixed heads are within 0.1 points on both networks without training any exit parameters. Against BP, jointly trained exits are 3.3 points more accurate on MobileNetV1 at similar cost and 2.6 points more accurate on VGG8 at 0.52 instead of 0.68 of the MACs, mainly because the BP networks are more accurate to begin with. Among stopping rules, a simple margin threshold matches our sync rule on most partitions, since the heads are already calibrated (ECE ≤ 0.08).

Table 2: Early exit at $\delta = 1\%$ on CIFAR10. MACs: fraction of the full network. LLS partitions: $(4, 2, 6, 2)$ for MobileNetV1, $(3, 3, 1)$ for VGG8.

	MobileNetV1		VGG8	
Exits	Acc. (%)	MACs	Acc. (%)	MACs
Fixed LLS heads (ours)	82.3	0.10	89.2	0.68
Linear probes on LLS	82.2	0.08	89.3	0.67
BP, frozen exits	82.8	0.10	92.1	0.70
BP, jointly trained	85.6	0.09	91.8	0.52

4.3 Scalability

We compare each LLS model with BP on the same network (Table 3). On ResNet-20, per-block LLS reaches 51.4%, 36.9 points below BP, and LLS-MxM improves on it by 4.6 points. Merging the blocks into three

resolution-aligned groups helps much more: $(4, 3, 3)$ reaches 71.9%, which improves on per-block LLS-MxM by 15.8 points and uses 40% less memory than BP, but is still 16.4 points below BP. On Wide-ResNet-20, $(7, 3)$ reaches 89.8%, 2.5 points below BP, with 12.7% less memory and 3% fewer training MACs than BP; it is not faster in wall-clock time (22.9 vs 22.2 s per epoch).

Table 3: Scalability on CIFAR10 (seed 0). Memory: analytic peak activation memory. The BP and per-block runs on ResNet-20 stopped early (patience 10).

Network	Method	Epochs	Acc. (%)	Memory (MB)
ResNet-20	BP	48	88.3	281.5
	Per-block LLS	57	51.4	56.5
	Per-block LLS-MxM	57	56.1	56.5
	$(4, 3, 3)$ LLS-MxM	200	71.9	170.0
Wide-ResNet-20	BP	300	92.4	561.6
	$(7, 3)$ LLS-MxM, LLS readout	300	89.8	490.0

4.4 Attention

We compare the six-block LLS-MxM ViT-Tiny with BP on the same network (Table 4). It reaches 77.9%, 3.1 points below BP, with 74% less measured GPU memory than BP, but its epochs take 19% longer. LLS therefore transfers to self-attention with a small accuracy cost and the largest memory saving in this project.

Table 4: Attention on CIFAR10 (ViT-Tiny, 100 epochs, seed 0). Memory: measured peak GPU memory.

Method	Acc. (%)	Memory (MB)	Time (s/epoch)
BP	81.0	1368.2	19.6
$(2, 2, 2, 2, 2, 2)$ LLS-MxM	77.9	353.3	23.3

4.5 Conclusions

A partition determines both the training memory and the exits available at inference, and merging the early layers improves both. The LLS heads are also interpretable in a practical sense: they are calibrated, so their confidence is a reliable exit signal; their per-layer accuracy shows where the network stops improving, which tells us where to merge layers; and the depth at which a sample exits follows its difficulty [12]. The same lesson holds on ResNets and ViTs: merging units into a few blocks recovers most of the accuracy lost by per-unit LLS, bringing Wide-ResNet-20 and ViT-Tiny within about 3 points of BP. LLS does not outperform BP, either as a classifier or as a source of exits, so its benefit is lower training memory. Since the best partition is model-specific, a natural next step is to search it automatically, as SEDONA [9] does for locally trained blocks.

References

[1] M. P. E. Apolinario, A. Roy, K. Roy. LLS: Local learning rule for deep neural networks inspired by neural activity synchronization. WACV 2025. https://arxiv.org/abs/2405.15868

[2] LLS reference implementation. https://github.com/mapolinario94/LLS-DNN

[3] T. P. Lillicrap et al. Random synaptic feedback weights support error backpropagation for deep learning. Nature Communications, 2016. https://arxiv.org/abs/1411.0247

[4] A. Nøkland. Direct feedback alignment provides learning in deep neural networks. NeurIPS 2016. https://arxiv.org/abs/1609.01596

[5] C. Frenkel, M. Lefebvre, D. Bol. Learning without feedback: fixed random learning signals allow for feedforward training of deep neural networks. Frontiers in Neuroscience, 2021.

[6] G. Hinton. The forward-forward algorithm: some preliminary investigations, 2022. https://arxiv.org/abs/2212.13345

[7] A. Nøkland, L. H. Eidnes. Training neural networks with local error signals. ICML 2019.

[8] E. Belilovsky, M. Eickenberg, E. Oyallon. Decoupled greedy learning of CNNs. ICML 2020.

[9] M. Pyeon, J. Moon, T. Hahn, G. Kim. SEDONA: Search for decoupled neural networks toward greedy block-wise learning. ICLR 2021.

[10] S. Teerapittayanon, B. McDanel, H. T. Kung. BranchyNet: Fast inference via early exiting from deep neural networks. ICPR 2016.

[11] Y. Kaya, S. Hong, T. Dumitras. Shallow-deep networks: Understanding and mitigating network overthinking. ICML 2019.

[12] R. J. N. Baldock, H. Maennel, B. Neyshabur. Deep learning through the lens of example difficulty. NeurIPS 2021.

[13] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. CVPR 2016. `https://arxiv.org/abs/1512.03385`

[14] A. Dosovitskiy et al. An image is worth 16x16 words: Transformers for image recognition at scale. ICLR 2021. `https://arxiv.org/abs/2010.11929`

[15] A. Defazio et al. The road less scheduled, 2024. `https://arxiv.org/abs/2405.15682`